

COMP 3069

Computer Graphics



Lecture 3:
Graphics Programming, OpenGL
& GLSL
Autumn 2025



The University of
Nottingham

Contents

- ◆ Making a window (GLFW)
- ◆ Loading OpenGL methods (GLAD)
- ◆ Compiling and linking a shader program (OpenGL)
- ◆ Rasterising using the barycentric coordinates (C++)
- ◆ Using VBO and VAO (OpenGL)
- ◆ Recognising a simple render loop (OpenGL)
- ◆ Simple vertex shader and fragment shader (GLSL)
- ◆ Passing variables between rendering stages (GLSL)
- ◆ Specifying vertex shader input variables (GLSL)
- ◆ Interface matching (GLSL)
- ◆ Using a GLSL uniform (GLSL)
- ◆ Deinitialisation (GLFW)



Making a Window

- ◆ OpenGL does not create a window
- ◆ A window is specific to the operating system
- ◆ OpenGL is designed to work on any operating system
- ◆ A programmer needs an alternative library for creating a window 替代库
- ◆ A library, called **Graphics Library Framework (GLFW)**, is used to create a window 图形库框架

Essentials of Making a Window

- ◆ Initialise GLFW 在使用任何 GLFW 函数之前，必须先调用：glfwInit()
- ◆ Specify OpenGL version 指定OpenGL版本
- ◆ Create a window 使用 GLFW 提供的函数创建一个实际的窗口对象：GLFWwindow* window = glfwCreateWindow(w, h, title, NULL, NULL);
- ◆ Specify that drawing should be to the window 指定渲染目标
- ◆ Specify a window resizing function 指定窗口大小变化的响应函数

```
9  GLFWwindow* CreateWindow(int w, int h, const char* title)
10 {
11     glfwInit(); 初始化 GLFW
12     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3); 指定 OpenGL 主版本号
13     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3); 指定 OpenGL 次版本号
14     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE); 使用核心模式
15     GLFWwindow* window = glfwCreateWindow(w, h, title, NULL, NULL); 创建窗口
16     glfwMakeContextCurrent(window); 指定渲染目标的代码行：绑定上下文（让 OpenGL
17     glfwSetFramebufferSizeCallback(window, framebuffer_size_callback); 知道在哪个窗口绘制)
18     return window; 指定窗口大小变化的响应函数
19 }
20
```

Loading OpenGL Methods

- ◆ It is necessary to load each of the OpenGL methods that a programmer wants to use at run-time, before the methods can be used
- ◆ This process is automated using GLAD
- ◆ In this course, we will use

```
gladLoadGLLoader((GLADloadproc)glfwGetProcAddress);
```

让 GLAD 使用 glfwGetProcAddress 这个函数，去加载所有的 OpenGL 函数地址。

to load all of the OpenGL methods

Create and Compile Vertex Shaders

- ◆ Use OpenGL to create a shader 着色器
- ◆ Specify the source code
- ◆ Use OpenGL to compile the shader
- ◆ Check for errors

```
10 unsigned int vertexShader = glCreateShader(GL_VERTEX_SHADER); 创建着色器
11 char* vertexShaderSource = read_file(vertexShaderFile); 读顶点着色器源码, 保存完整
12 glShaderSource(vertexShader, 1, &vertexShaderSource, NULL); shader源代码
13 glCompileShader(vertexShader);
14 glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &success); 查询函数: 检查shader是否编译
15 if (!success) 成功
16 { 如果失败, 执行{}中内容, 并且获取编译错误信息
17     glGetShaderInfoLog(vertexShader, 512, NULL, infoLog);
18     std::cout << "ERROR::SHADER::VERTEX::COMPILATION_FAILED\n" << infoLog << std::endl;
19     } 打印错误提示和具体错误日志
```

Create and Compile Fragment Shaders

- ◆ The process used to create, compile a vertex shader etc. are also used for a fragment shader
像素着色器

```
21 unsigned int fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
22 char* fragmentShaderSource = read_file(fragmentShaderFile);
23 glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
24 glCompileShader(fragmentShader);
25 glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &success);
26 if (!success)
27 {
28     glGetShaderInfoLog(fragmentShader, 512, NULL, infoLog);
29     std::cout << "ERROR::SHADER::FRAGMENT::COMPILATION_FAILED\n" << infoLog << std::endl;
30 }
..
```

Create a Shader Program

- ◆ A shader program is used to combine the shaders for use in the rendering process 渲染进程
- ◆ CPU程序 Use OpenGL to create a shader program
把前面编译好的 vertex shader 和 fragment shader 附加到这个程序上。
- ◆ Attach the shaders to the program
- ◆ Use OpenGL to link the program

```
32      unsigned int shaderProgram = glCreateProgram();
33      glAttachShader(shaderProgram, vertexShader);
34      glAttachShader(shaderProgram, fragmentShader);
35      glLinkProgram(shaderProgram);
--
```

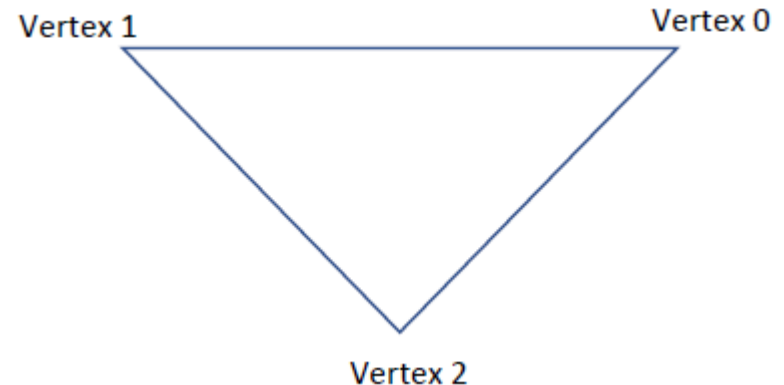


Specifying Vertices

- ◆ Each vertex has vertex attributes which describe the vertex 每一个顶点 (vertex) 都有一些 “顶点属性 (vertex attributes)” 来描述它
- ◆ At the very least a vertex needs a 3-dimensional Euclidean position 至少每个顶点必须有一个三维欧几里得坐标。
额外的顶点属性可以包括：颜色、法向量和纹理坐标。
- ◆ Additional attributes may include a vertex colour, a vertex normal, and a texture coordinate
- ◆ These attributes are interpolated by the rasteriser using the barycentric coordinates 重心坐标插值
这些属性在光栅化 (rasterization) 过程中会被用 “重心坐标 (barycentric coordinates)” 插值。

假设三个顶点有颜色：

- Vertex 0: 红色 (1,0,0)
- Vertex 1: 绿色 (0,1,0)
- Vertex 2: 蓝色 (0,0,1)



当 GPU 画出三角形时，它会在每个像素点上根据位置比例自动混合出中间颜色，比如紫色、青色、黄绿色等。
这就是“插值”。

Specifying Vertex Positions

- ◆ The programmer specifies the vertices in the application as **an array**. 使用数组来指定顶点坐标
- ◆ **A triangle** made of 3 vertices with only a position of each vertex can **be specified as**

```
6      float vertices[] =
7      {
8          //pos
9          0.5f, 0.5f, 0.f,
10         -0.5f, 0.5f, 0.f,
11         -0.f, -0.5f, 0.f,
12     };
```

坐标说明 (OpenGL 默认坐标系):

- 原点 (0, 0) 在屏幕中心;
- x轴向右为正;
- y轴向上为正;
- z轴朝屏幕外为正 (这里 $z = 0$ 表示二维平面)。

vertices

0.5

0.5

0

-0.5

0.5

0

-0

-0.5

0



Setting Up Objects

◆ In order to render, it is necessary to have

一组顶点数据（带有属性），用于描述物体的几何形状

- a list of vertices described by attributes

一个顶点着色器，用来处理每一个顶点。

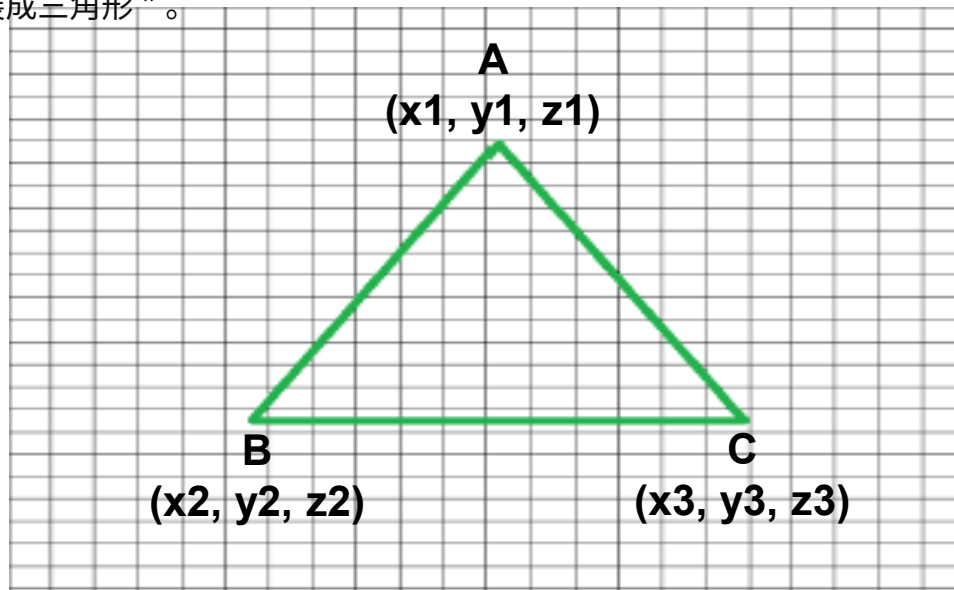
- a vertex shader which process each vertex

一个片段着色器，用来计算每个像素（fragment）的颜色

- a fragment shader which calculates the colour of the fragment

◆ The list of vertices is used to assemble triangles

顶点列表会被 GPU 用来“拼装成三角形”。



VBO & VAO

顶点着色器 (vertex shader) 需要逐个访问每个顶点属性。

- ◆ The vertex shader will need to access each vertex attribute individually

OpenGL 使用两个对象来指定顶点数组是如何被顶点着色器访问的。

- ◆ Two OpenGL objects are set to specify how the vertex arrays are accessed by the vertex shader

顶点缓冲对象：存放顶点属性数组 它的作用就是：把 CPU 的数组 (float vertices[]) 复制到 GPU 显存中。

- Vertex buffer object (VBO) contains the vertex attribute array

顶点数组对象：保存OpenGL绘制所需要的所有顶点属性配置

- Vertex array object (VAO) contains all of the detail necessary for OpenGL to render

VAO 并不存储具体数据，而是记录顶点属性的结构和布局。

对象	全称	功能	示例比喻
VBO	Vertex Buffer Object	存储顶点数据 (坐标、颜色等)	“食材仓库”
VAO	Vertex Array Object	记录如何读取顶点数据 (结构说明)	“菜单 + 操作说明书”
Shader	顶点 / 片段着色器	处理顶点和像素数据	“厨师”



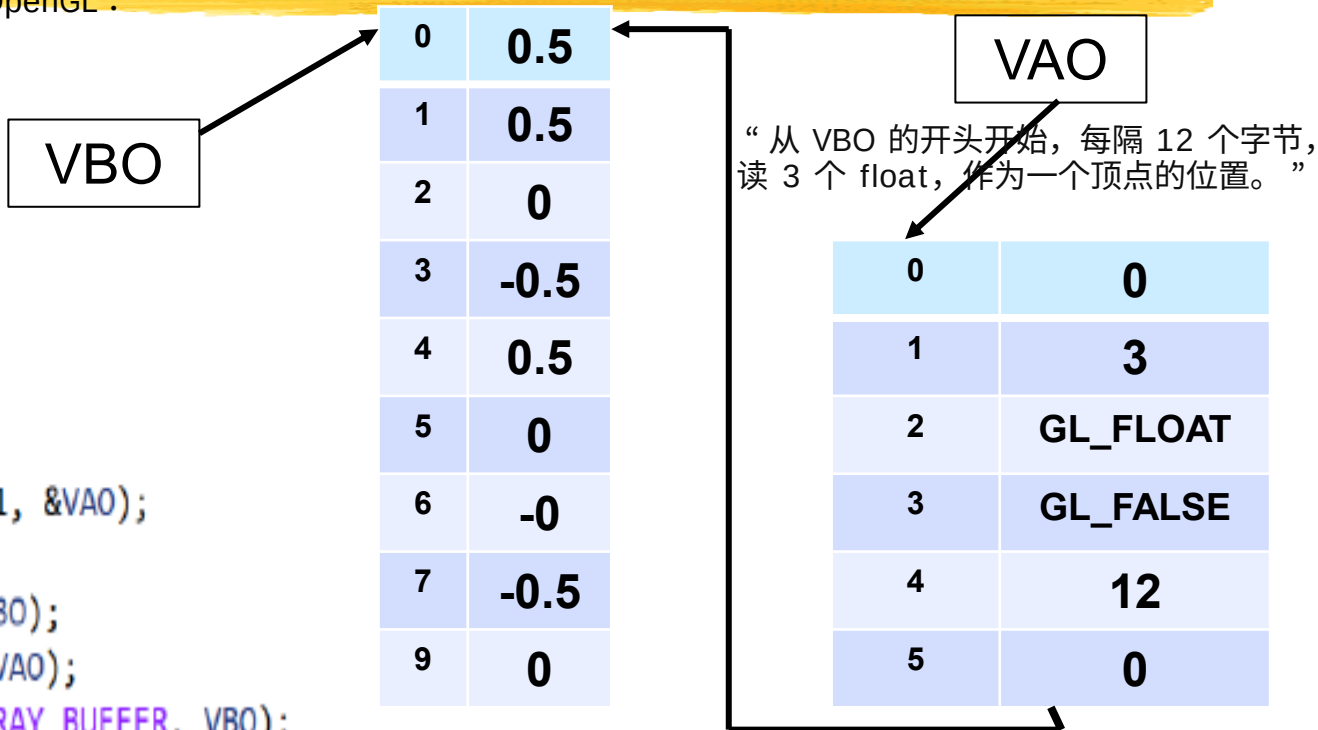
VBO 存放顶点数据，VAO 记录如何读取这些数据。
GPU 最终通过 VAO 知道该从哪个 VBO 中，以什么格式去取顶点坐标

Setting Up VBO & VAO

左边蓝表 (VBO) : 是顶点数据数组 (0.5, 0.5, 0 ...);

右边蓝表 (VAO) : 是配置表, 告诉 OpenGL:

“ 这些数据每 3 个 float 是一个顶点,
每个 float 是 GL_FLOAT 类型,
不要归一化,
步长是 12 字节,
从偏移 0 开始读取。 ”



```
104 unsigned int VAO;  
105 glGenVertexArrays(1, &VAO);  
106 unsigned int VBO;  
107 glGenBuffers(1, &VBO);  
108 glBindVertexArray(VAO);  
109 glBindBuffer(GL_ARRAY_BUFFER, VBO);  
110 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);  
111 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);  
112 glEnableVertexAttribArray(0);  
113 glBindBuffer(GL_ARRAY_BUFFER, 0);  
114 glBindVertexArray(0);
```

VAO specifies how VBO should be used



Render Loop

```
1. while(window should stay open)
{
    2. process input
    3. clear the screen
    4. draw
    5. swap buffers
}
```



Code of Render Loop

```
120 while (!glfwWindowShouldClose(window))
121 {
122     processInput(window);
123
124     glClearColor(1.f, 1.f, 1.f, 1.f);
125     glClear(GL_COLOR_BUFFER_BIT );
126
127
128     glUseProgram(shaderProgram);
129     glBindVertexArray(VAO);
130     glDrawArrays(GL_TRIANGLES, 0, 3);
131     glBindVertexArray(0);
132
133     glfwSwapBuffers(window);
134     glfwPollEvents();
135 }
```



Closing Window

渲染循环 (Render Loop) 应在用户关闭窗口时自动停止。

- ◆ The render loop should stop executing when the user has closed the window with the close flag as

```
int glfwWindowShouldClose  
(GLFWwindow * window)
```

- ◆ At the very least, processing input should check if the user wants to close the window as

```
40 void processInput(GLFWwindow* window)
41 {
42
43
44     if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
45     {
46         glfwSetWindowShouldClose(window, true);
47     }
48 }
```


Deinitialisation GLFW 反初始化

为什么要“反初始化”？

OpenGL 程序退出时，必须释放 GLFW 分配的所有资源，否则会导致内存泄漏或系统资源占用。

- ◆ GLFW needs to be **deinitialised** using

```
void glfwTerminate (void)
```

- ◆ This method destroys the windows created
会自动销毁所有创建的窗口；

138

```
glfwTerminate();
```

OpenGL Shading Language GLSL

- ◆ The OpenGL Shading Language (GLSL) is used to program shaders 它是用于编写 Shader（着色器）的语言。
- ◆ A shader is a program which runs on the graphics processing unit (GPU) GLSL 是运行在 GPU 上的程序语言；

类型	处理对象	功能
Vertex Shader	每个顶点	计算顶点坐标、空间变换（模型→世界→相机→投影）
Fragment Shader	每个像素	决定最终颜色、光照、纹理等效果

#version : 指明版本 ;
in / out : 输入输出变量 ;
uniform : 全局常量 (如矩阵、光照参数) ;

More about Shaders

layout(location = x) : 绑定属性编号 (与 glVertexAttribPointer 对应) 。

- ◆ A shader resembles a C program, with a `main()` method.
- ◆ The shader also includes additional lines like the `#version` directive and additional syntax like the `out` qualifier.
- ◆ GLSL, like C, has common operators e.g. '+' and '*' and common functions like 'cos()'.
- ◆ We will use vertex shaders and fragment shaders.
- ◆ A vertex shader processes a single vertex
- ◆ A fragment shader processes a single fragment

Vertex Shader

- ◆ Vertex Shader processes individual vertices
- ◆ The most simple vertex shader will just copy the vertex position, as it was defined in the vertex specification, and pass it as an output to the next stage without any change
- ◆ Primary use of the vertex shader is to transform vertices
- ◆ Vertex shader has input vertex attributes, which represent an input vertex
- ◆ Vertex shader then outputs one vertex. 每次只输出一个顶点



Sample of Vertex Shader

```
1  #version 330 core
2
3  layout(location = 0) in vec3 aPos;
4
5  void main()
6  {
7      gl_Position = vec4(aPos, 1.f);
8  }
9  |
```

Fragment Shader

- ◆ The Fragment Shader processes individual fragments
- ◆ The primary use of the fragment shader is to compute the colour of pixels
- ◆ The fragment shader has input vertex attributes that were output by the vertex shader, which have been interpolated to the fragment position, together representing a fragment
- ◆ The fragment shader then outputs a fragment colour
- ◆ The most simple fragment shader will specify a solid colour and pass it out to be stored in the colour buffer 只返回一个固定颜色



Sample of Fragment Shader

- ◆ This fragment shader outputs red for the fragment colour

```
1  #version 330 core
2
3  out vec4 fragColour;
4
5  void main()
6  {
7      fragColour = vec4(1.f, 0.f, 0.f, 1.0f);
8  }
9
```

GLSL Comments

- ◆ Comments can be written in the same way as in C. A single-line comment looks like

```
//vec3 pos;
```

- ◆ A multi-line comment looks like

```
/*vec3 pos;
```

```
vec3 col;*/
```


GLSL Variables

- ◆ Variables can be declared for use in a shader
- ◆ We will use the following data types

Data Type	
float	A floating-point number
vec2, vec3, vec4	A vector of 2, 3, 4 floating-point numbers
mat4	A matrix with 4 columns and 4 rows
sampler2D	A sampler for accessing 2D texture colours

数据类型

含义

示例

float

单个浮点数

```
float brightness = 0.8;
```

vec2 / vec3 / vec4

向量 (2、3、4个浮点数)

```
vec3 pos = vec3(1.0, 0.5, 0.0);
```

mat4

4×4 矩阵, 用于空间变换 (如投影矩阵)

```
mat4 model;
```

sampler2D

用于访问 2D 纹理

```
uniform sampler2D texture1;
```



Type Qualifiers

- ◆ If no type qualifier is declared then the variable will be default
- ◆ A shader has input and output variables and can also access textures and uniforms
- ◆ If a variable is declared with `in` qualifier, then it is an input variable for that shader, e.g.,

```
in vec4 pos;
```

- ◆ If a variable is declared with `out` qualifier then it is an output variable for that shader, e.g.,

```
out vec2 tex;
```

GLSL Built-in Variables

- ◆ All built-in variables start with the prefix `gl_`. Non-built-in variables cannot be declared to start with `gl_`
所有内建变量都以 `gl_` 开头
- ◆ For example `gl_Position` is a built-in variable. It is the output of the vertex shader and is the position of the current vertex

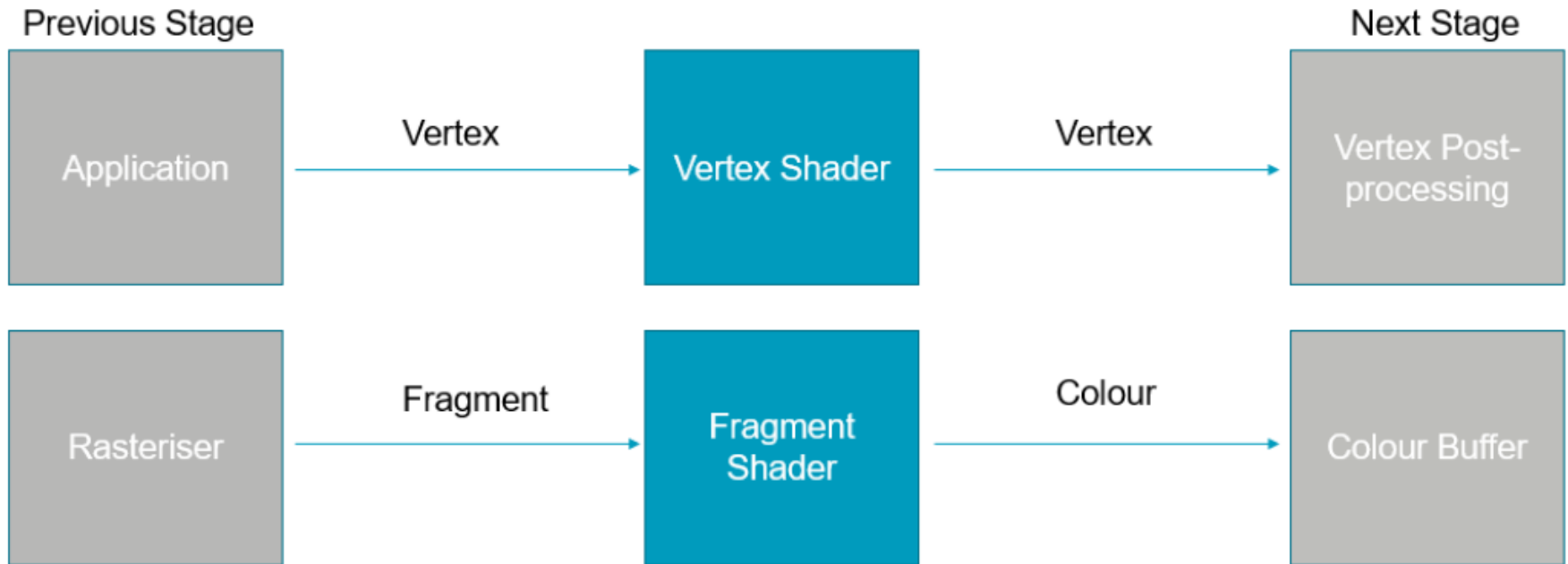
变量名	所属阶段	含义
<code>gl_Position</code>	顶点着色器	顶点位置输出 (必须)
<code>gl_FragCoord</code>	片段着色器	当前像素的屏幕坐标
<code>gl_PointSize</code>	顶点着色器	绘制点图元的大小
<code>gl_VertexID</code>	顶点着色器	当前顶点的索引号
<code>gl_FrontFacing</code>	片段着色器	是否为正面多边形

Shader Input & Output Variables

- ◆ Shader input variables are passed from the previous stage in the rendering pipeline 渲染管线
- ◆ In the case of the vertex shader, the input variable is passed from the application stage in the form of a vertex
- ◆ In the case of the fragment shader, the input variable is passed from the rasterisation stage in the form of a fragment
- ◆ Shader output variables are passed on to the next stage in the rendering pipeline
- ◆ In the case of the vertex shader, the output variable is passed to the Vertex Post Processing stage in the form of a vertex
- ◆ In the case of the fragment shader, the output variable is passed to the colour buffers in the form of a colour.

PU 读取顶点数据 每个顶点生成颜色 插值生成像素 每个像素着色 绘制红色三角形。

Illustration of Shader Inputs & Outputs



顶点着色器接收顶点 计算每个顶点在屏幕上的位置；
光栅化器把这些顶点变成像素 片段着色器计算每个像素的颜色 输出到屏幕。

Array of Vertex Attribute

- ◆ Specifying vertex shader input variables in GLSL is carried out in unison with the OpenGL code for specifying each attribute in the vertex array
- ◆ The array of vertex attribute values are defined in application C code, which looks something like this

```
58  
59 float vertices[] =  
60 {  
61     //pos           //col  
62     0.f,  0.5f, 0.f,  1.f, 0.f, 0.f,  
63     -0.5f, -0.5f, 0.f, 0.f, 1.f, 0.f,  
64     0.5f,  -0.5f, 0.f, 0.f, 0.f, 1.f,  
65 };
```

Each Attribute in Vertex Array

- ◆ Each attribute in the vertex array is set up in the VAO

```
67 unsigned int VAO;
68 glGenVertexArrays(1, &VAO);
69 unsigned int VBO;
70 glGenBuffers(1, &VBO);
71 glBindVertexArray(VAO);
72 glBindBuffer(GL_ARRAY_BUFFER, VBO);
73 glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
74 glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
75 glEnableVertexAttribArray(0);
76 glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 * sizeof(float)));
77 glEnableVertexAttribArray(1);
78 glBindBuffer(GL_ARRAY_BUFFER, 0);
79 glBindVertexArray(0);
```

Specify position attribute values (not colour attribute values)

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
```



Location of Vertex Shader Input

- ◆ The location of vertex shader input variables are specified in the vertex shader
- ◆ The first vertex attribute, first input variable (`location = 0`) to the vertex shader is a `vec3` and represents the position (`pos`) of the vertex.
- ◆ The second vertex attribute, the second input variable (`location = 1`) to the vertex shader is a `vec3` and represents the colour (`col`) of the vertex.

```
1  #version 330 core
2
3  layout(location = 0) in vec3 aPos;
4  layout(location = 1) in vec3 aCol;
5
6  out vec3 col;
```



Interface Matching

- ◆ The output of a vertex shader must match the input of the fragment shader, in terms of the interface
- ◆ Interface matching is checked when the shader program is linked
- ◆ In order to match, the variables must have the same name and type
- ◆ In the example above, we output the vertex colour from the vertex shader
- ◆ This must match the input to the fragment shader

```
1  #version 330 core
2
3  in vec3 col;
4
```

uniform 是一种全局变量，
它的值在 CPU 设置一次后，可以在着色器中多次使用（不会随顶点或像素变化）

GLSL Uniforms

- ◆ A uniform is a variable that can have its value set by the application program before the GLSL program is executed
- ◆ Its value will not change for all executions of the shaders in the program
- ◆ Uniforms are declared with the uniform qualifier
- ◆ If a uniform value is modified in the shader code, then the shader will not compile



Assigning Value to Uniform

- ◆ The way that the application program changes the value of the uniform is first to find the location of the uniform and then to set the value

```
95      glUseProgram(shaderProgram);
96
97      glm::vec3 col = glm::vec3(1.f, 0.6f, 0.f);
98      int loc = glGetUniformLocation(shaderProgram, "uCol");
99      glUniform3f(loc, col.x, col.y, col.z);
100
101      glBindVertexArray(VAO);
102      glDrawArrays(GL_TRIANGLES, 0, 6);
```

Declaring and Using Uniform

- ◆ Uniform can then be declared and used in the fragment shader

```
1  #version 330 core
2
3  in vec3 col;
4
5  out vec4 fragColour;
6
7  uniform vec3 uCol;
8
9  void main()
10 {
11     fragColour = vec4(uCol, 1.0f);
12 }
```



References

- ◆ *COMP3069 Lecture notes: Chapter 3 - Graphics Programming, OpenGL & GLSL*
- ◆ *Interactive Computer Graphics: A Top-Down Approach with Shader-Based OpenGL, Chapter 2*